

4. Automatización de redes con formatos estructurados

Objetivo general

Aplicar formatos estructurados como XML, JSON y YAML en la automatización de tareas de configuración, monitoreo y gestión de redes, integrándolos en herramientas, protocolos y flujos de trabajo colaborativos.

Sintaxis y estructura de XML, JSON y YAML

Comparativo técnico

Formato	Sintaxis	Ventajas	Desventajas
XML	Etiquetas anidadas	Estándar robusto, validación con XSD	Verboso, menos legible
JSON	Clave-valor con corchetes y llaves	Ligero, ampliamente usado en APIs	No soporta comentarios
YAML	Indentación jerárquica	Legible, ideal para configuración	Sensible a espacios, menos formal

Ejemplo de equivalencia:

XML

Xml

 Copiar

```
<interface>
  <name>eth0</name>
  <ip>192.168.1.1</ip>
</interface>
```

JSON

Json

 Copiar

```
{
  "interface": {
    "name": "eth0",
    "ip": "192.168.1.1"
  }
}
```

YAML

Yaml

 Copiar

```
interface:
  name: eth0
  ip: 192.168.1.1
```

Actividad práctica:

Tarea: Convertir una misma configuración de red a los tres formatos.

Producto: Archivos `.xml`, `.json` y `.yaml` validados y comentados.

Aplicación en protocolos y herramientas: NETCONF, RESTCONF, Ansible, Terraform

Aplicaciones clave:

Herramienta / Protocolo	Formato usado	Aplicación
NETCONF	XML	Configuración estructurada de dispositivos de red
RESTCONF	JSON / XML	Gestión de dispositivos vía HTTP/REST
Ansible	YAML	Automatización de tareas y configuraciones
Terraform	HCL / JSON	Infraestructura como código (IaC) para redes y servicios

Ejemplo de playbook en YAML (Ansible):

Yaml

Copiar

```
- name: Configurar interfaz de red
  hosts: routers
  tasks:
    - name: Asignar IP a eth0
      ios_config:
        lines:
          - interface eth0
          - ip address 192.168.1.1 255.255.255.0
```

Actividad práctica:

Tarea: Crear un playbook en YAML para configurar una interfaz en un router simulado.

Producto: Archivo `.yaml` funcional y ejecutado en entorno virtual (e.g., GNS3, Cisco Packet Tracer).

Validación de formatos: esquemas, linters y pruebas

Herramientas de validación:

Formato	Herramientas	Propósito
XML	XSD, xmllint	Validación estructural y semántica
JSON	JSON Schema, jsonlint	Validación de estructura y tipos
YAML	yamllint, PyYAML	Verificación de sintaxis e indentación

Buenas prácticas:

- Usar validadores antes de desplegar configuraciones
- Documentar esquemas para facilitar la interoperabilidad
- Integrar validaciones en pipelines CI/CD

Actividad práctica:

Tarea: Validar un archivo JSON y YAML con herramientas online o de línea de comandos.

Producto: Reporte de validación + correcciones aplicadas.